# LAPORAN PRAKTIKUM 4

# PEMROGRAMAN BERORIENTASI OBJEK

# EKSPANSI PRODUK KEUANGAN (INHERITANCE)


Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

NURFIAH, S.ST, M.KOM

Asisten Praktikum :

EZZA ADDINI

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS


2026

# DAFTAR ISI

**DAFTAR ISI**.....i

**BAB 1 PENDAHULUAN**

1.1 Latar Belakang..... 1

1.2 Tujuan Praktikum.....1

1.3 Manfaat Praktikum.....2

**BAB 2 PEMBAHASAN**

2.1 Tahapan Pelaksanaan Praktikum.....3

2.1.1 Membuat Package dan Class..... 3

2.1.2 Membuat Class Rekening.....3

2.1.3 Membuat Class Transaksi.....7

2.1.4 Membuat Class Main.....8

2.1.5 Membuat Class RekeningTabungan..... 13

2.1.6 Membuat Class RekeningGiro.....14

2.2 Tugas Praktikum..... 16

2.2.1 Tugas 1.....16

2.2.2 Tugas 2.....18

2.2.3 Output Tugas 1 dan Tugas 2..... 19

**BAB 3 KESIMPULAN**

3.1 Kesimpulan..... 20

3.2 Link Repository.....20

# BAB I

# PENDAHULUAN

## 1.1 Latar Belakang

Dalam pemrograman berorientasi objek, sebuah program dapat memiliki beberapa class yang memiliki sifat atau fungsi yang saling berkaitan. Jika setiap class dibuat dari awal, akan terdapat banyak kode yang sama dan membuat program menjadi kurang efisien. Oleh karena itu, konsep inheritance (pewarisan) digunakan agar sebuah class dapat mewarisi atribut dan method dari class lain sehingga prinsip code reusability dapat diterapkan. Pada Praktikum 4 ini, konsep tersebut diterapkan pada sistem perbankan dengan Rekening sebagai superclass serta RekeningTabungan dan RekeningGiro sebagai subclass.

Selain memahami hubungan antara superclass dan subclass, praktikum ini juga menerapkan keyword extends dan super() dalam proses pewarisan. Program kemudian dikembangkan menggunakan upcasting dan downcasting untuk mengelola berbagai jenis rekening dalam satu koleksi. Dengan penerapan tersebut, mahasiswa dapat memahami bagaimana inheritance digunakan untuk membuat program yang lebih terstruktur dan mengurangi pengulangan kode.

## 1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami prinsip Code Reusability melalui Pewarisan (Inheritance).
2. Membedakan Superclass dan Subclass.
3. Menggunakan keyword extends untuk mewariskan sifat, serta super() untuk memanggil konstruktor induk.
4. Menerapkan Upcasting dan Downcasting pada koleksi objek.

## 1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami penerapan code reusability melalui konsep inheritance pada program Java.
2. Mampu membedakan superclass dan subclass serta memahami hubungan keduanya.
3. Memahami penggunaan extends dan super() dalam proses pewarisan class.
4. Memahami penerapan upcasting dan downcasting untuk mengelola objek dari beberapa subclass dalam satu koleksi.

# BAB II

# PEMBAHASAN

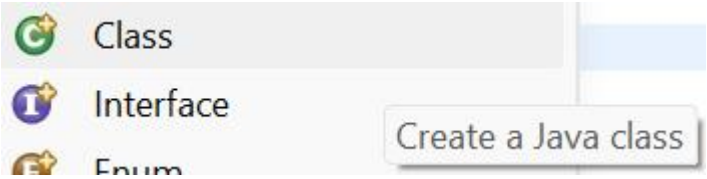
## 2.1 Tahapan Pelaksanaan Praktikum

### 2.1.1 Membuat Package dan Class

1. Buat Package dengan nama Praktikum4.

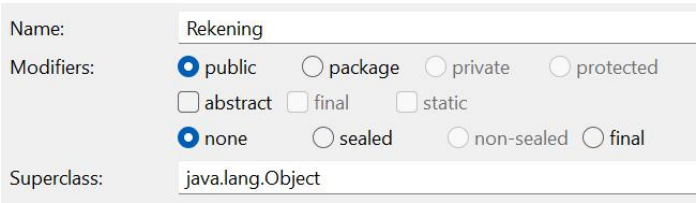



2. Setelah membuat Package, lanjutkan dengan membuat Class.




### 2.1.2 Membuat Class Rekening

1. Buat class baru dengan nama Rekening pada package Praktikum4. Pada class ini nantinya akan dibuat data dan method yang berhubungan dengan sebuah rekening.




2. Scanner dan ArrayList digunakan untuk kebutuhan input dan penyimpanan data rekening pada program.

```
package Praktikum1;
import java.util.Scanner;
import java.util.ArrayList;
```

3. Selanjutnya, buat atribut rekening dengan menggunakan access modifier private.

```
// Bagian atas dari class Rekening.java
 private String nomorRekening;
    private String namaPemilik;
    private String pin;
```

Penggunaan private bertujuan untuk membatasi akses langsung dari luar class terhadap data rekening. Atribut pin juga dibuat private karena merupakan data yang digunakan untuk keamanan rekening.

4. Kemudian, ubah access modifier pada atribut saldo dan riwayatTransaksi menjadi protected.

```
// Gunakan protected agar Subclass bisa mengaksesnya langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;
```

saldo dan riwayatTransaksi diubah menjadi protected agar dapat digunakan dan dimanipulasi oleh class turunan seperti RekeningTabungan dan RekeningGiro. Sementara itu, pin dan nomorRekening tetap menggunakan private untuk menjaga keamanan data.

5. Selanjutnya buat constructor dengan nama yang sama seperti nama class. Constructor ini menerima 4 parameter, yaitu nomor, nama, saldoAwal dan pinAwal.

```
// Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, double saldoAwal,
    String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
```

6. Setelah data rekening diinisialisasi, tambahkan proses validasi PIN pada constructor.

```
// Validasi PIN di dalam Constructor
if (pinAwal.length() == 6) {
    this.pin = pinAwal;
} else {
    System.out.println("Peringatan: PIN harus 6 digit!
    Menggunakan PIN default 123456");
    this.pin = "123456";
```

Kode digunakan untuk memeriksa panjang PIN yang dimasukkan saat rekening dibuat. Jika panjang PIN adalah 6 karakter, PIN tersebut digunakan. Jika tidak, program memberikan peringatan dan menggunakan PIN default 123456 .

7. Setelah itu, inisialisasi riwayatTransaksi menggunakan ArrayList.

```
this.riwayatTransaksi = new ArrayList<>();
     riwayatTransaksi = new ArrayList<>();
     System.out.println("Rekening atas nama " + namaPemilik + "
     berhasil dibuat.");
```

riwayatTransaksi diinisialisasi agar dapat digunakan untuk menyimpan transaksi yang dilakukan oleh rekening.

8. Buat getter untuk atribut yang masih diperbolehkan untuk dibaca dari luar class.

```
// Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }
```

Getter digunakan agar data nomorRekening dan namaPemilik tetap dapat diakses oleh class lain tanpa membuka akses langsung terhadap atribut private.

9. Buat method otentikasi() untuk melakukan pemeriksaan PIN.

```
// Method Otontikasi Internal (Validasi Enkapsulasi)
    public boolean otentikasi(String inputPin) {
       return this.pin.equals(inputPin);
```

Method otentikasi() digunakan untuk membandingkan PIN yang tersimpan pada rekening dengan PIN yang dimasukkan oleh pengguna. Jika PIN sesuai, method mengembalikan nilai true, sedangkan jika PIN tidak sesuai akan mengembalikan nilai false.

10. Buat method setorTunai()

```
public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
            nominal);
            riwayatTransaksi.add(trxBaru);
```

```
            System.out.println("Setor tunai Rp" + nominal + "
            berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih
            dari 0!");
        }
    }
```

Method setorTunai() digunakan untuk melakukan penyetoran uang ke rekening. Program mengecek apakah nominal setor lebih dari 0, kemudian menambahkan nominal tersebut ke saldo. Jika berhasil, transaksi dicatat sebagai Kredit dan disimpan ke dalam riwayatTransaksi.

11. Buat method tarikTunai()

```
public void tarikTunai(double nominal) {
            if (nominal < 10000) {
              System.out.println("Transaksi Gagal : Minimal
            nomonal penarikan 10.000");
            } else if (nominal > saldo) {
              System.out.println("Transaksi Gagal: Saldo tidak
            mencukupi. Saldo Anda: Rp" + saldo);
            } else { saldo -= nominal;
              System.out.println("Tarik tunai berhasil.");
              System.out.println("Jumlah penarikan: Rp" + nominal);
              System.out.println("Saldo sekarang: Rp" + saldo);

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit",
            nominal);
            riwayatTransaksi.add(trxBaru);
        }
    }
```

Method tarikTunai() digunakan untuk melakukan penarikan uang dari rekening. Program mengecek apakah nominal penarikan sudah memenuhi batas minimal dan saldo mencukupi. Jika penarikan berhasil, saldo akan dikurangi dan transaksi dicatat sebagai Debit ke dalam riwayatTransaksi.

12. Buat method cekInformasi()

```
public void cekInformasi() {
        System.out.println("=== INFO REKENING ===");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("====================");
```

```
}
```

Method cekInformasi() digunakan untuk menampilkan informasi rekening yang sedang aktif. Informasi yang ditampilkan meliputi nomor rekening, nama pemilik, dan saldo akhir rekening.

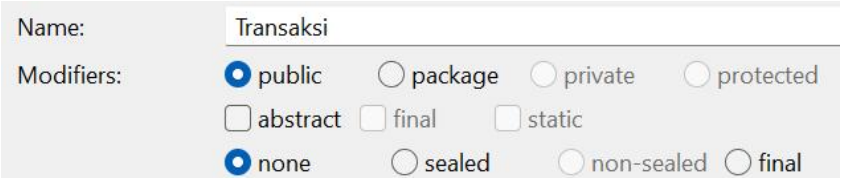
13. Buat method cetakMutasi()

```
public void cetakMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening
            ini");
        } else {
            for (Transaksi transaksi : riwayatTransaksi) {
                transaksi.cetakDetail();
}
```

Method cetakMutasi() digunakan untuk menampilkan riwayat transaksi pada rekening. Program mengecek apakah terdapat transaksi, kemudian menggunakan perulangan for-each untuk menampilkan detail setiap transaksi melalui method cetakDetail(). Jika belum ada transaksi, program akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut.

### 2.1.3 Membuat Class Transaksi

1. Buat class baru dengan nama Transaksi pada package Praktikum4.




2. Tambahkan tiga atribut yaitu idTransaksi untuk menyimpan identitas atau ID transaksi, jenis untuk menyimpan jenis transaksi seperti Debit atau Kredit, dan nominal untuk menyimpan jumlah uang yang terlibat dalam transaksi.

```
public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;
```

3. Selanjutnya, buat constructor yang digunakan untuk menginisialisasi data transaksi saat objek Transaksi dibuat. Setiap objek transaksi memiliki ID, jenis transaksi, dan jumlah transaksi yang sesuai.

```
// Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
```

4. Setelah itu, buat method cekDetail() yang digunakan untuk menampilkan detail transaksi ke layar.

```
public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " +
        jenis + " | Nominal: Rp" + nominal);
}
```

### 2.1.4 Membuat Class Main

1. Membuat scanner dan variabel program.

```
public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null; // Objek belum diinisialisasi
        (null)
        boolean isRunning = true;
        String pinYangDiInput;
```

Kode di atas digunakan untuk menyiapkan input pengguna dengan Scanner, menyimpan daftar rekening menggunakan ArrayList, menentukan akun yang sedang aktif melalui akunAktif, dan mengatur program agar tetap berjalan dengan isRunning. Variabel pinYangDiInput digunakan untuk menyimpan PIN yang dimasukkan pengguna saat proses autentikasi.

2. Membuat menu utama. Perulangan while digunakan agar menu terus ditampilkan selama program masih berjalan.

```
System.out.println("=== SISTEM PERBANKAN MINI ===");

while (isRunning) {
      System.out.println("\n=== Menu Utama ===");
      System.out.println("1. Buka Rekening Baru");
      System.out.println("2. Setor Tunai");
      System.out.println("3. Tarik Tunai");
      System.out.println("4. Cek Informasi Rekening");
      System.out.println("5. Ganti Akun");
      System.out.println("6. Cetak Mutasi (Riwayat)");
      System.out.println("0. Keluar");
      System.out.print("Pilih menu: ");
```

3. Input dari pengguna disimpan ke dalam variabel pilihan. Selanjutnya, switch digunakan untuk menentukan proses yang dijalankan sesuai dengan nomor menu yang dipilih.

```
int pilihan = input.nextInt();
       input.nextLine(); // Membersihkan buffer enter

       switch (pilihan) {
```

4. Membuat proses buka rekening

```
case 1:
  System.out.print("Masukkan No Rekening: ");
  String no = input.nextLine();
  System.out.print("Masukkan Nama Pemilik: ");
  String nama = input.nextLine();
  System.out.print("Masukkan Saldo Awal: ");
  double saldo = input.nextDouble();
  input.nextLine();

  System.out.print("Masukkan PIN (6 digit): ");
  String pin = input.nextLine();

  Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
  daftarRekening.add(rekeningBaru);
  akunAktif = rekeningBaru;
  System.out.println("Rekening berhasil ditambahkan.");
  break;
```

Pada menu ini, pengguna memasukkan nomor rekening, nama pemilik, saldo awal, dan PIN. Setelah itu, data tersebut digunakan untuk membuat

objek Rekening. PIN diberikan ke constructor agar tersimpan pada rekening dan dapat digunakan untuk proses autentikasi.

5. Membuat proses setor tunai

```
case 2:

if (akunAktif == null) {
       System.out.println("Error: Mohon maaf, Anda belum memiliki
       nomor rekening!");
 } else {
       System.out.print("Masukkan nominal setor: ");
       double setor = input.nextDouble();
       akunAktif.setorTunai(setor);
       }
 break;
```

Sebelum melakukan setor tunai, program mengecek apakah sudah ada rekening yang aktif. Jika belum ada, program menampilkan pesan kesalahan. Jika sudah ada, nominal setor dimasukkan dan method setorTunai() dipanggil.

6. Membuat proses tarik tunai dengan autentikasi PIN

```
case 3:
  if (akunAktif == null) {
      System.out.println("Error: Anda belum membuka rekening!");
  } else {
      System.out.print("Masukkan PIN: ");
      pinYangDiInput = input.nextLine();

      if (akunAktif.otentikasi(pinYangDiInput)) {
       System.out.print("Masukkan nominal tarik: Rp");
       double tarik = input.nextDouble();
       akunAktif.tarikTunai(tarik);
      } else {
      System.out.println("Akses Ditolak: PIN yang Anda masukkan
      salah!");
      }
  }
  break;
```

Pada menu tarik tunai, program terlebih dahulu mengecek apakah terdapat akun aktif. Jika ada, pengguna diminta memasukkan PIN. PIN tersebut kemudian diperiksa menggunakan method otentikasi()

7. Membuat proses cek informasi

```
case 4:

if (akunAktif == null) {
    System.out.println("Error: Anda belum membuka rekening!");
} else {
    akunAktif.cekInformasi();
}
break;
```

Jika ada akun aktif, program menjalankan cekInformasi() untuk menampilkan data rekening.

8. Membuat proses mengganti akun aktif

```
case 5 :
    if (daftarRekening.isEmpty()) {
        System.out.println("Belum ada rekening yang tersedia.");
    } else {
        System.out.print("Masukkan nomor rekening yang ingin digunakan: ");
        String nomorCari = input.nextLine();
        boolean ditemukan = false;
        for (Rekening rekening : daftarRekening) {
            if (rekening.getNomorRekening().equals(nomorCari)) {
                akunAktif = rekening;
                ditemukan = true;
                System.out.println("Berhasil mengganti akun.");
                System.out.println("Akun aktif: "+ akunAktif.getNamaPemilik());
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("Rekening tidak ditemukan.");
        }
    }
    break;
```

Pengguna memasukkan nomor rekening, kemudian program mencari rekening tersebut di daftarRekening dan menjadikannya sebagai akun aktif. Digunakan method getNomorRekening() dan getNamaPemilik() karena atribut pada class Rekening sudah dibuat private.

9. Membuat proses cetak mutasi dengan autentikasi PIN

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        pinYangDiInput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiInput)) {
        akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Case 6 digunakan untuk menampilkan riwayat transaksi atau mutasi rekening. Sebelum cetakMutasi() dijalankan, pengguna harus memasukkan PIN terlebih dahulu. Jika PIN benar, riwayat transaksi ditampilkan. Jika PIN salah, akses ditolak dan riwayat transaksi tidak ditampilkan. Ketentuan ini sesuai dengan tugas modul yang meminta autentikasi sebelum menu 6 dijalankan.

10. Membuat proses menghentikan program

```
case 0:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;

default:
        System.out.println("Pilihan tidak valid!");
```

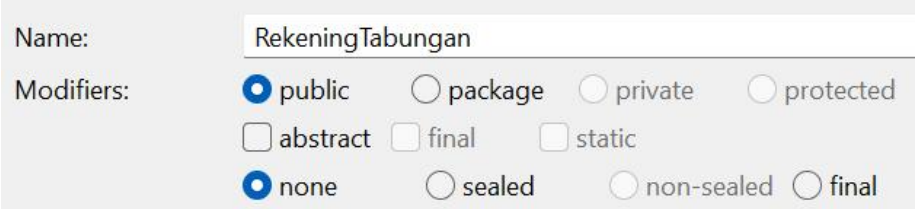
Default digunakan untuk menampilkan pesan kesalahan jika pengguna memasukkan pilihan menu yang tidak tersedia.

11. Program diakhiri dengan menutup scanner

```
input.close();
```

### 2.1.5 Membuat Class RekeningTabungan

1. Buat file baru bernama RekeningTabungan sebagai subclass dari Rekening.




2. RekeningTabungan menggunakan extends untuk mewarisi atribut dan method yang terdapat pada class Rekening. Dengan begitu, kode yang sudah ada pada Rekening dapat digunakan kembali tanpa membuatnya dari awal.

```
public class RekeningTabungan extends Rekening {
```

3. Tambahkan atribut khusus untuk rekening tabungan yaitu suku bunga bulanan.

```
// Atribut spesifik yang hanya dimiliki oleh Tabungan
 private double sukuBunga;
```

Atribut sukuBunga digunakan untuk menyimpan nilai bunga yang dimiliki oleh rekening tabungan. Fitur ini menjadi tambahan khusus pada RekeningTabungan.

4. Kemudian, buat constructor RekeningTabungan dengan menggunakan super() untuk memanggil constructor dari class Rekening.

```
// Constructor Subclass
public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
    // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
    super(nomor, nama, saldoAwal, pinAwal);
    this.sukuBunga = sukuBunga;
}
```

super() digunakan untuk meneruskan data rekening seperti nomor rekening, nama pemilik, saldo awal, dan PIN ke constructor superclass Rekening. Pemanggilan super() harus berada di awal constructor subclass.

5. Selanjutnya, Tambahkan method untuk menghitung bunga akhir bulan.

```
    public void tambahBungaAkhirBulan() {
        // Menghitung bunga
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

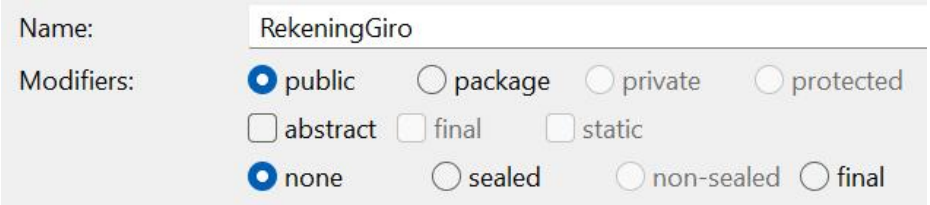
        // Mencatat riwayat transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }
}
```

Method tersebut digunakan untuk menghitung bunga berdasarkan saldo rekening dan persentase sukuBunga, kemudian hasil bunga ditambahkan ke saldo rekening.

### 2.1.6 Membuat Class RekeningGiro

1. Buat file baru bernama RekeningGiro sebagai subclass dari Rekening.




RekeningGiro dibuat sebagai class turunan dari Rekening, sehingga dapat mewarisi fitur yang sudah ada pada class induk.

2. Tambahkan atribut khusus berupa batas overdraft.

```
public class RekeningGiro extends Rekening {
    private double batasOverdraft;
```

batasOverdraft digunakan untuk menyimpan batas pinjaman yang dapat digunakan pada rekening giro. Fitur ini menjadi karakteristik tambahan dari RekeningGiro.

3. Selanjutnya, buat constructor RekeningGiro, gunakan super() untuk memanggil constructor dari Rekening.

```
public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
    // Memanggil inisialisasi dasar dari Superclass
    super(nomor, nama, saldoAwal, pinAwal);
    this.batasOverdraft = batasOverdraft;
}
```

super() digunakan untuk menginisialisasi data dasar rekening dari class Rekening, sedangkan batasOverdraft digunakan untuk menyimpan data khusus rekening giro.

4. Setelah itu, buat getter untuk mengakses nilai batasOverdraft.

```
// Getter khusus Giro
public double getBatasOverdraft() {
	return batasOverdraft;
}
```

Getter digunakan untuk mengambil nilai batasOverdraft dari objek RekeningGiro tanpa mengakses atribut private secara langsung.

## 2.2 Tugas Praktikum

### 2.2.1 Tugas 1 (Pembuatan Multi-Produk)

Lakukan perubahan pada Menu 1 atau proses buka rekening dengan menambahkan pilihan produk setelah pengguna memasukkan saldo awal dan PIN. Pengguna dapat memilih Tabungan Umum atau Giro Bisnis, kemudian memasukkan data tambahan sesuai produk yang dipilih, yaitu suku bunga untuk Tabungan atau batas overdraft untuk Giro. Objek rekening yang dibuat kemudian disimpan ke dalam daftarRekening menggunakan konsep upcasting.

- **Kode Program :**

```
System.out.println("Pilih Produk:");
System.out.println("1. Tabungan Umum");
System.out.println("2. Giro Bisnis");
System.out.print("Pilihan: ");
int pilihanProduk = input.nextInt();
input.nextLine();

if (pilihanProduk == 1) {
    System.out.print("Masukkan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    Rekening rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;

} else if (pilihanProduk == 2) {
    System.out.print("Masukkan Batas Overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();

    Rekening rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
}

System.out.println("Rekening berhasil ditambahkan.");
break;
```

- **Langkah Kerja :**

1. Buka class Main dan cari bagian case 1 yang digunakan untuk proses membuka rekening baru.
2. Setelah user memasukkan saldo awal dan PIN, ditambahkan pilihan produk rekening.

```
System.out.println("Pilih Produk:");
System.out.println("1. Tabungan Umum");
```

```
System.out.println("2. Giro Bisnis");
System.out.print("Pilihan: ");
int pilihanProduk = input.nextInt();
input.nextLine();
```

Kode tersebut digunakan untuk memberikan pilihan kepada user untuk menentukan jenis rekening yang akan dibuat, yaitu Tabungan Umum atau Giro Bisnis.

3. Jika user memilih Tabungan Umum, program meminta input suku bunga dan membuat objek RekeningTabungan menggunakan data rekening yang sudah dimasukkan sebelumnya.

```
if (pilihanProduk == 1) {
    System.out.print("Masukkan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    Rekening rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
```

4. Jika user memilih Giro Bisnis, program meminta input batas overdraft dan membuat objek RekeningGiro.

```
} else if (pilihanProduk == 2) {
    System.out.print("Masukkan Batas Overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();

    Rekening rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
}
```

5. Selanjutnya, simpan objek yang dibuat ke dalam daftarRekening menggunakan referensi bertipe Rekening.

```
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

### 2.2.2 Tugas 2 (Simulasi Waktu Akhir Bulan)

Tambahkan menu Simulasi Akhir Bulan (Khusus Tabungan) pada Main.java. Pada menu tersebut, sistem memeriksa jenis akun aktif menggunakan instanceof, kemudian melakukan downcasting dan menjalankan method tambahBungaAkhirBulan() jika akun merupakan RekeningTabungan. Jika akun bukan rekening Tabungan, proses ditolak dan sistem menampilkan pesan bahwa fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

- **Langkah Kerja :**

1. Tambahkan menu baru pada daftar menu utama untuk menjalankan simulasi akhir bulan.

```
System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
```

Menu 7 ditambahkan untuk menjalankan fitur simulasi akhir bulan yang hanya dapat digunakan oleh rekening Tabungan.

2. Buat case 7 dan cek apakah akun aktif merupakan objek RekeningTabungan menggunakan instanceof.

```
case 7:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else if (akunAktif instanceof RekeningTabungan) {
```

Operator instanceof digunakan untuk memeriksa apakah akun aktif merupakan objek RekeningTabungan sebelum fitur bunga akhir bulan dijalankan.

3. Setelah dipastikan bahwa akun aktif merupakan RekeningTabungan, objek tersebut diubah ke tipe RekeningTabungan menggunakan downcasting. Selanjutnya method tambahBungaAkhirBulan() dipanggil untuk menambahkan bunga akhir bulan.

```
RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
tabungan.tambahBungaAkhirBulan();
```

4. Tambahkan kondisi jika akun aktif bukan merupakan RekeningTabungan.

```
} else {
    System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
}
break;
```

Jika akun aktif bukan rekening Tabungan, sistem tidak menjalankan proses bunga dan menampilkan pesan bahwa fitur tersebut hanya berlaku untuk Rekening Tabungan.

### 2.2.3 Output Tugas 1 dan Tugas 2

```
=== SISTEM PERBANKAN MINI ===

=== Menu Utama ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 7185982
Masukkan Nama Pemilik: Kinaya
Masukkan Saldo Awal: 7100000
Masukkan PIN (6 digit): novrya
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 5
Rekening atas nama Kinaya berhasil dibuat.
Rekening berhasil ditambahkan.

=== Menu Utama ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Bunga 5.0% berhasil ditambahkan: Rp355000.0
```

Output tersebut menunjukkan bahwa user berhasil membuat rekening Tabungan Umum dengan saldo awal Rp7.100.000 dan suku bunga 5%. Setelah memilih Menu 7, sistem mengenali akun sebagai rekening Tabungan dan menjalankan simulasi akhir bulan. Bunga sebesar Rp355.000 berhasil dihitung dari 5% saldo awal dan ditambahkan ke saldo rekening.

# BAB III
# KESIMPULAN

## 3.1 Kesimpulan

Pada Praktikum 4 ini, telah diterapkan konsep inheritance dengan menjadikan Rekening sebagai superclass dan membuat RekeningTabungan serta RekeningGiro sebagai subclass. Dengan konsep ini, fitur yang sudah ada pada Rekening dapat digunakan kembali oleh class turunannya, sementara masing-masing jenis rekening dapat memiliki fitur tambahan seperti suku bunga pada Tabungan dan batas overdraft pada Giro.

Selain itu, program juga telah dikembangkan agar dapat menangani beberapa jenis produk rekening melalui upcasting dan downcasting. Pada pengujian, rekening Tabungan berhasil dibuat dengan suku bunga dan simulasi akhir bulan dapat dijalankan menggunakan instanceof, kemudian dilakukan downcasting untuk memanggil tambahBungaAkhirBulan(). Dengan demikian, praktikum ini membantu memahami penerapan inheritance serta penggunaan kembali kode dalam pengembangan program Java.

## 3.2 Link Repository

https://github.com/kinayanovryamanda7-blip/PrakPBO_B_2511531016